

课程笔记

# CS224N Lecture 4: Dependency Parsing

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年 1 月

**课程版本：**2024 年中文讲义整理版

**笔记来源：**AI Course Notes Contributors

**本版处理：**移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

**授权：**笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

### 来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture04>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

# 目录

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1 引言：为什么需要句法结构</b>                         | <b>3</b>  |
| 1.1 本章小结                                      | 3         |
| <b>2 句法结构的两种视角</b>                            | <b>3</b>  |
| 2.1 短语结构语法 (Phrase Structure Grammar)         | 3         |
| 2.2 依存结构 (Dependency Structure)               | 4         |
| 2.3 本章小结                                      | 4         |
| <b>3 句法歧义：理解语言的核心挑战</b>                       | <b>4</b>  |
| 3.1 介词短语附着歧义 (PP Attachment Ambiguity)        | 4         |
| 3.2 并列结构作用域歧义 (Coordination Scope Ambiguity)  | 5         |
| 3.3 修饰语附着歧义                                   | 5         |
| 3.4 歧义的实际影响                                   | 5         |
| 3.5 句法结构在 NLP 中的应用                            | 6         |
| 3.6 本章小结                                      | 6         |
| <b>4 依存语法详解</b>                               | <b>6</b>  |
| 4.1 依存语法的基本概念                                 | 6         |
| 4.2 依存语法的悠久历史                                 | 7         |
| 4.3 箭头方向的约定                                   | 7         |
| 4.4 依存分析的决策依据                                 | 7         |
| 4.5 投射性与非投射性 (Projectivity)                   | 8         |
| 4.6 本章小结                                      | 8         |
| <b>5 标注数据与树库 (Treebanks)</b>                  | <b>8</b>  |
| 5.1 从规则到数据驱动                                  | 8         |
| 5.2 Universal Dependencies (UD)               | 9         |
| 5.3 本章小结                                      | 9         |
| <b>6 基于转移的依存分析 (Transition-based Parsing)</b> | <b>9</b>  |
| 6.1 依存分析的方法概览                                 | 9         |
| 6.2 Arc-Standard 转移系统                         | 9         |
| 6.3 实例演示：分析 “I ate fish”                      | 10        |
| 6.4 Nivre 与贪心决策                               | 10        |
| 6.5 传统特征工程方法                                  | 11        |
| 6.6 本章小结                                      | 11        |
| <b>7 神经网络依存分析器</b>                            | <b>11</b> |
| 7.1 从离散特征到分布式表示                               | 11        |
| 7.2 Chen & Manning (2014) 神经依存分析器             | 11        |
| 7.3 三种分布式表示的作用                                | 12        |
| 7.4 非线性分类器的优势                                 | 12        |
| 7.5 实验结果                                      | 12        |

|           |                                        |           |
|-----------|----------------------------------------|-----------|
| 7.6       | 本章小结                                   | 13        |
| <b>8</b>  | <b>依存分析的评估</b>                         | <b>13</b> |
| 8.1       | UAS 与 LAS                              | 13        |
| 8.2       | 本章小结                                   | 13        |
| <b>9</b>  | <b>后续发展：从 SyntaxNet 到图方法</b>           | <b>14</b> |
| 9.1       | Google SyntaxNet / Parsey McParseface  | 14        |
| 9.2       | 神经图方法依存分析 (Neural Graph-based Parsing) | 14        |
| 9.3       | 本章小结                                   | 14        |
| <b>10</b> | <b>总结与延伸</b>                           | <b>15</b> |
| 10.1      | 讲者的核心总结                                | 15        |
| 10.2      | 全课知识图谱                                 | 15        |
| 10.3      | 关键 Takeaways                           | 16        |
| 10.4      | 拓展阅读                                   | 16        |

# 1 引言：为什么需要句法结构

当我们阅读或听到一句话时，表面上是一个线性的词序列（或线性的声音流），但人类大脑会自动地将其组织为具有层次结构的意义单元——判断哪些词修饰哪些词、哪些词组成一个短语。自然语言处理（NLP）系统同样需要理解句子的内部结构，才能正确地解释语言。

## 本讲核心内容

1. **句法结构的两种表示**：短语结构（Constituency/Phrase Structure）与依存结构（Dependency Structure）
2. **依存语法的基本概念**：中心词（head）与依存词（dependent）、依存关系类型
3. **句法歧义**：介词短语附着歧义、并列结构作用域歧义等
4. **依存句法分析方法**：基于转移的分析（Transition-based Parsing）、基于图的分析（Graph-based Parsing）
5. **神经网络依存分析器**：用分布式表示替代离散特征，实现更高精度与更快速度

本讲内容直接关联课程作业 Assignment 2 的第二部分——使用 PyTorch 构建一个神经依存句法分析器。

## 1.1 本章小结

句法结构是理解自然语言的基础。人类在理解语言时自动地解析句子的层次结构，NLP 系统也需要具备这种能力。本讲将从语言学概念出发，逐步引入计算方法。

# 2 句法结构的两种视角

## 2.1 短语结构语法（Phrase Structure Grammar）

语言学中表示句子结构的第一种方式是**短语结构**（Phrase Structure），由上下文无关文法（Context-Free Grammar, CFG）来形式化描述。

### 上下文无关文法（CFG）基本概念

上下文无关文法通过一组**重写规则**描述语言的结构。每条规则将一个非终结符展开为终结符或非终结符的序列。例如：

- $NP \rightarrow Det (Adj)^* N$ （名词短语 = 限定词 + 可选的形容词 + 名词）
- $PP \rightarrow P NP$ （介词短语 = 介词 + 名词短语）
- $VP \rightarrow V PP$ （动词短语 = 动词 + 介词短语）
- $NP \rightarrow NP PP$ （名词短语也可以递归包含介词短语）

以短语 “the cuddly cat by the door” 为例，其短语结构树自底向上构建：

- “the cuddly cat” 构成一个名词短语（NP）

- “the door” 构成另一个名词短语
- “by the door” 构成一个介词短语 (PP)
- 整体 “the cuddly cat by the door” 是一个更大的名词短语

## 2.2 依存结构 (Dependency Structure)

本讲的重点是第二种表示方式——**依存结构** (Dependency Structure)。依存结构不关心短语的层次嵌套，而是直接刻画**词与词之间的修饰关系**。

### 依存结构的核心思想

依存结构通过**有向弧** (directed arcs) 连接句子中的词对，表示**中心词** (head) 与**依存词** (dependent/modifier) 之间的关系。例如在 “look in the large crate in the kitchen by the door” 中：

- “look” 是整个句子的中心词 (root)
- “crate” 是 “in” 的宾语，“in the large crate” 修饰 “look”
- “large” 和 “the” 修饰 “crate”
- “in the kitchen” 和 “by the door” 也修饰 “crate”

## 2.3 本章小结

句法结构有两种主要表示方式：短语结构（基于上下文无关文法的树形嵌套）和依存结构（基于词间修饰关系的有向图）。两者可以相互转化，但本课程重点使用依存结构，因为它更直接地表达词与词之间的语义关系，且在 NLP 实践中更为流行。

## 3 句法歧义：理解语言的核心挑战

人类语言与编程语言的一个根本区别在于：**人类语言存在全局歧义 (global ambiguity)**，而编程语言的歧义总能通过语法规则唯一消解。

### 3.1 介词短语附着歧义 (PP Attachment Ambiguity)

以新闻标题 “Scientists count whales from space” 为例：

- **解读 1**：“from space” 修饰 “count” ——科学家**从太空**数鲸鱼（正确解读）
- **解读 2**：“from space” 修饰 “whales” ——科学家数**来自太空的**鲸鱼

这种歧义源于英语中介词短语 (PP) 可以附着到不同的中心词上。由于 PP 在英语中极为常见，这类歧义也极为普遍。

### PP 附着歧义的数量爆炸

当句子中有多个连续的介词短语时，可能的解读数量按 **Catalan 数** 增长（指数级）。Manning 举了 Wall Street Journal 的例子：“The board approved its acquisition by Royal Trustco Ltd. of Toronto for \$27 a share at its monthly meeting” 包含 4 个连续 PP，理论上 有 14 种可能的依存结构（实际由上下文和世界知识消歧）。

## 3.2 并列结构作用域歧义 (Coordination Scope Ambiguity)

另一类常见歧义来自**并列结构** (coordination) 的作用域。例如：

“Shuttle veteran and longtime NASA executive Fred Gregory appointed to board”

- **解读 1**：Fred Gregory 是一个人，既是 shuttle veteran 又是 NASA executive
- **解读 2**：有两个人——一个 shuttle veteran 和 NASA executive Fred Gregory

类似的歧义还出现在 “Doctor: No heart, cognitive issues” 中——“No” 的作用域是仅覆盖 “heart” 还是同时覆盖 “heart” 和 “cognitive issues”。

## 3.3 修饰语附着歧义

不定式短语 “to be used for Olympics beach volleyball” 可以修饰 “beach”（海滩将用于沙滩排球）或 “body”（尸体将用于沙滩排球），产生截然不同的含义。

### 不同语言的歧义类型不同

Manning 特别指出，不同语言的句法歧义类型并不完全相同。例如，中文的介词短语修饰动词时通常出现在动词之前，宾语在动词之后，因此不会出现英语中 PP 修饰动词 vs. 修饰宾语的歧义。但中文有自己独特的歧义类型。

## 3.4 歧义的实际影响

Catalan 数列决定了 PP 附着歧义的组合爆炸速度：

| PP 数量 | 可能的解读数 |
|-------|--------|
| 1     | 2      |
| 2     | 5      |
| 3     | 14     |
| 4     | 42     |
| 5     | 132    |

表 1: PP 数量与可能解读数的关系 (Catalan 数)

尽管歧义数量指数增长，人类读者通常能毫不费力地选择正确解读——这依赖于世界知识、上下文和语用推理。NLP 系统需要从数据中学会类似的消歧能力。

### 3.5 句法结构在 NLP 中的应用

依存结构在实际 NLP 系统中有广泛应用。例如在生物信息学领域，可以通过依存分析从文本中提取蛋白质-蛋白质相互作用事实。动词 “interacts” 的主语和介词宾语分别指向两个交互的蛋白质，并列结构则帮助识别多重交互关系。

### 3.6 本章小结

句法歧义是自然语言的根本特征，包括 PP 附着歧义、并列结构作用域歧义和修饰语附着歧义等。歧义数量随句子复杂度指数增长，但人类能利用世界知识和上下文轻松消歧。NLP 系统需要从标注数据中学习这种消歧能力。理解并正确解析句法结构对信息抽取、语义理解等下游任务至关重要。

## 4 依存语法详解

### 4.1 依存语法的基本概念

依存语法的核心元素：

#### 依存语法的形式化定义

- **依存关系 (Dependency)**：一个有向弧，从**中心词** (head/governor) 指向**依存词** (dependent/modifier)
- **依存树 (Dependency Tree)**：所有依存关系构成一棵有向树，有唯一的根节点 (root)
- **关系标签 (Relation Labels)**：每条弧上标注语法关系类型，如 nsubj (名词主语)、obj (宾语)、amod (形容词修饰语)、det (限定词)、obl (斜格) 等
- **虚拟根节点 (ROOT)**：通常在句子前添加一个虚拟的 ROOT 节点，其唯一依存词是句子的主要动词

## 4.2 依存语法的悠久历史

### 依存语法的历史脉络

依存语法是人类历史上最古老的句法形式化方法：

- **Panini** (约公元前 4-8 世纪)：最早的语法学家，用依存关系描述梵语 (Sanskrit) 的语法结构。他的语法最初以**口述方式**传承数百年，令人惊叹
- **阿拉伯语法传统** (第一千年)：使用依存语法描述阿拉伯语
- **Reed-Kellogg 句子图解**：在美国教育中广泛使用的句子图解法，本质上是依存语法的一种变体
- **Lucien Tesnière** (20 世纪)：现代依存语法的奠基人，确立了从中心词指向依存词的箭头方向
- 相比之下，**短语结构语法/CFG** 是非常晚近的发明 (1940s–1950s，由 Chomsky 等人系统化)

Manning 特别提到了一个有趣的事实：计算机科学家通常从 Chomsky 层次结构了解 Chomsky，但 Chomsky 层次结构最初并非为了形式语言理论——而是为了论证人类语言不能用正则/有限状态文法来充分描述。

## 4.3 箭头方向的约定

在依存语法中，箭头方向存在两种约定：

- **Head → Dependent**：Tesnière 传统，也是本课程使用的方向
- **Dependent → Head**：另一些研究者使用的方向

### 注意箭头方向约定

在阅读不同的依存语法文献时，必须首先确认所使用的箭头方向约定。不同的工具和标注方案可能采用不同的约定，混淆方向会导致对结构的完全误读。

## 4.4 依存分析的决策依据

Manning 总结了依存分析器在做出依存决策时需要考虑的四类信息：

1. **词汇亲和性 (Bilexical Affinities)**：两个词之间是否存在合理的依存关系。例如 “discussion ← issues” 是合理的，但 “the ← completed” 则不合理
2. **依存距离 (Dependency Distance)**：大多数依存关系发生在**相邻或近距离**的词之间。长距离依存虽然存在，但较为少见
3. **中间成分 (Intervening Material)**：依存弧很少跨越动词或标点符号
4. **中心词配价 (Valency of Heads)**：中心词能接受多少个依存词。例如 “broke” 通常有一个主语（左侧）和一个可选的宾语（右侧），但不能接受多个宾语

## 4.5 投射性与非投射性 (Projectivity)

### 投射性与非投射性依存

- **投射性 (Projective)**: 依存弧不交叉, 等价于上下文无关语法可生成的结构。大多数依存关系满足投射性
- **非投射性 (Non-projective)**: 依存弧存在交叉。例如 “I’ll give a talk tomorrow on neural networks” 中, “on neural networks” 修饰 “talk”, “tomorrow” 修饰 “give”, 两条弧交叉

疑问句中的 Wh-移位也常产生非投射依存: “Who did Bill buy the coffee from yesterday?” 中 “who” 是 “from” 的宾语, 但被移到了句首。

本课程的作业中只处理投射性依存分析。

## 4.6 本章小结

依存语法通过中心词-依存词关系描述句子结构, 具有悠久的历史传统。依存树具有唯一根节点和树形结构约束。分析器的决策依赖词汇亲和性、距离偏好、中间成分和配价约束。大多数依存关系是投射性的, 但自然语言中也存在非投射性的情况。

## 5 标注数据与树库 (Treebanks)

### 5.1 从规则到数据驱动

早期的 NLP 系统依赖**手写规则**来解析句子结构。这种方法存在两个致命问题:

1. **覆盖率不足**: 人类语言具有无穷的创造性 (如 Yoda 式语序重排、网络用语等), 手写规则难以穷举
2. **无法消歧**: 纯规则系统可以生成所有合法的句法分析结果, 但无法在多个合法分析之间选择最可能的那个

为了解决这些问题, NLP 社区从 1980 年代末到 1990 年代开始大规模建设**标注语料库** (annotated corpora), 即**树库** (Treebanks) ——由人工标注了句法结构的句子集合。

#### 树库为何重要

树库的建设虽然缓慢而费力, 但带来了两大革命性优势:

1. **可复用的训练数据**: 一次标注, 多次使用。可以训练句法分析器、词性标注器、心理语言学模型等
2. **系统化的评估方法**: 提供了客观的基准测试。在树库出现之前 (1950s–1980s), NLP 系统的评估方式是: 运行程序, 输入一句话, 看看输出的分析结果是否 “看起来合理” ——没有任何系统化的定量评估

## 5.2 Universal Dependencies (UD)

Manning 介绍了他深度参与的 **Universal Dependencies (UD)** 项目——一个统一标注方案的跨语言依存树库：

- 覆盖超过 **100 种语言**
- 使用统一的依存关系标签体系
- 适用于跨语言研究、心理语言学实验等
- 课程作业 Assignment 2 使用的就是这类数据

## 5.3 本章小结

树库的建设推动了 NLP 从规则驱动转向数据驱动。Universal Dependencies 项目提供了覆盖 100 多种语言的标准化依存树库，是现代依存分析研究和应用的基础资源。树库同时提供了训练数据和评估基准。

# 6 基于转移的依存分析 (Transition-based Parsing)

## 6.1 依存分析的方法概览

依存分析有多种算法范式：

1. **动态规划方法** (Dynamic Programming)
2. **基于图的方法** (Graph-based / Maximum Spanning Tree)
3. **约束满足方法** (Constraint Satisfaction)
4. **基于转移的方法** (Transition-based / Shift-Reduce) ——本讲重点

基于转移的方法在实践中最为流行，因为它具有**线性时间复杂度**，远优于上下文无关文法分析的立方时间复杂度。

## 6.2 Arc-Standard 转移系统

基于转移的依存分析使用两个核心数据结构和三种操作：

## 转移系统的组成

### 数据结构：

- **栈 (Stack)**：存放正在处理的词，初始只包含 ROOT。**栈顶在右侧**
- **缓冲区 (Buffer)**：存放尚未处理的词，初始包含句子的所有词。**缓冲区顶在左侧**

### 三种操作：

1. **SHIFT**：将缓冲区顶部的词移到栈顶
2. **LEFT-ARC**：栈顶第二个词成为栈顶词的依存词（左弧），弹出第二个词
3. **RIGHT-ARC**：栈顶词成为栈顶第二个词的依存词（右弧），弹出栈顶词

**终止条件**：缓冲区为空，栈中只剩 ROOT。

## 6.3 实例演示：分析 “I ate fish”

以句子 “I ate fish” 为例，分析过程如下：

| 步骤 | 栈                 | 缓冲区            | 操作                                 |
|----|-------------------|----------------|------------------------------------|
| 0  | [ROOT]            | [I, ate, fish] | 初始状态                               |
| 1  | [ROOT, I]         | [ate, fish]    | SHIFT                              |
| 2  | [ROOT, I, ate]    | [fish]         | SHIFT                              |
| 3  | [ROOT, ate]       | [fish]         | LEFT-ARC (I $\leftarrow$ ate)      |
| 4  | [ROOT, ate, fish] | []             | SHIFT                              |
| 5  | [ROOT, ate]       | []             | RIGHT-ARC (ate $\rightarrow$ fish) |
| 6  | [ROOT]            | []             | RIGHT-ARC (ROOT $\rightarrow$ ate) |

表 2: “I ate fish” 的转移操作序列

最终构建的依存关系集合为：

- ate  $\rightarrow$  I (主语)
- ate  $\rightarrow$  fish (宾语)
- ROOT  $\rightarrow$  ate (句子根节点)

## 操作选择决定了分析结果

不同的操作序列会生成不同的依存结构。如果在第 3 步选择 RIGHT-ARC 而非 LEFT-ARC，就会让 “I” 成为中心词而 “ate” 成为依存词——这显然是错误的。因此，**如何在每一步选择正确的操作**是整个系统的核心问题。

## 6.4 Nivre 与贪心决策

瑞典 NLP 研究者 **Joakim Nivre** (2003–2005) 提出了用**机器学习**来预测每一步应采取的操作。  
核心洞察：

### 贪心转移分析的关键发现

1. 每步操作都可以视为一个**分类问题**：给定当前状态（栈、缓冲区内容），预测应执行 SHIFT、LEFT-ARC 还是 RIGHT-ARC
2. 尽管使用**贪心策略**（每步只选最优操作，不做搜索），分析精度仍然很高
3. 整个分析过程是**线性时间**  $O(n)$ ：每个词最多被 SHIFT 一次、参与一次 ARC 操作

## 6.5 传统特征工程方法

Nivre 最初（2005 年）使用的是传统的**符号化特征**（indicator features）配合线性分类器（如逻辑回归或 SVM）：

典型特征包括：

- 栈顶词是 “good” 且词性是形容词
- 栈顶词是 “good” 且栈中第二个词是 “had”
- 缓冲区顶部词的词性是名词

这些特征的组合可以达到数百万甚至更多。

### 传统特征方法的三大问题

1. **稀疏性** (Sparsity)：大多数特征组合在训练数据中极少出现，可能仅出现 10 次/百万句
2. **不完备性** (Incompleteness)：有些词和组合在训练数据中从未出现
3. **计算开销**：运行时分析中，**95% 以上的时间**花在计算特征上，而非机器学习决策本身

## 6.6 本章小结

基于转移的依存分析通过栈和缓冲区模拟分析过程，使用 SHIFT、LEFT-ARC、RIGHT-ARC 三种操作逐步构建依存树。Nivre 提出的贪心策略实现了线性时间复杂度。传统方法依赖大量手工特征，存在稀疏性、不完备性和高计算开销等问题。

## 7 神经网络依存分析器

### 7.1 从离散特征到分布式表示

针对传统特征方法的三大问题，自然的解决方案是使用**分布式表示**（distributed representations）——即我们在前几讲中学习的**词向量**（word embeddings）。

### 7.2 Chen & Manning (2014) 神经依存分析器

Danqi Chen（当时是 Manning 的博士生，也是 CS224N 的两届 Head TA）构建了第一个成功的**神经转移依存分析器**，其核心设计如下：

### 神经依存分析器的架构

**输入表示：**对于当前分析状态，提取以下关键元素的分布式表示并拼接 (concatenate)：

- 栈顶和缓冲区顶部的**词向量** (word embeddings)
- 对应位置的**词性向量** (POS tag embeddings)
- 已构建的依存弧的**关系标签向量** (dependency label embeddings)

**网络结构：**

$$h = \text{ReLU}(W_1 \cdot [\text{concatenated inputs}] + b_1)$$

$$\text{output} = \text{softmax}(W_2 \cdot h + b_2)$$

**输出：**一个概率分布，覆盖所有可能的操作 (SHIFT、各种标签的 LEFT-ARC、各种标签的 RIGHT-ARC)。

## 7.3 三种分布式表示的作用

### 为什么需要三种嵌入

- **词向量** (Word Embeddings)：捕获词汇语义相似性。即使训练数据中没有见过某个词在特定位置出现，如果该词与已见过的词语义相近，系统也能做出合理预测
- **词性向量** (POS Tag Embeddings)：将细粒度的词性标签（如单数名词 NN vs. 复数名词 NNS）映射到连续空间，使相似词性获得相近表示
- **依存标签向量** (Dependency Label Embeddings)：捕获已构建的依存关系类型之间的相似性，为后续决策提供结构化上下文

## 7.4 非线性分类器的优势

传统方法使用**线性分类器**（逻辑回归、SVM），而神经方法使用**非线性分类器**（含 ReLU 隐藏层的前馈网络）。这带来了两大优势：

1. **自动学习特征组合**：隐藏层能自动学习输入特征的非线性组合，不再需要手工设计特征交叉
2. **更强的表达能力**：非线性分类器能拟合更复杂的决策边界

## 7.5 实验结果

实验结果令人瞩目：

| 分析器                           | UAS (%) | LAS (%) | 速度       |
|-------------------------------|---------|---------|----------|
| MaltParser (Nivre, 转移 + 符号特征) | 89.9    | —       | 快        |
| MSTParser (图方法 + 符号特征)        | 91.5    | —       | 慢 (~50x) |
| Chen & Manning (转移 + 神经网络)    | 92.0    | —       | 快        |

表 3: 不同依存分析器的精度与速度对比

## 神经方法的双重突破

Chen & Manning (2014) 的神经依存分析器同时实现了：

1. **精度提升**：超越了传统的转移分析器 (MaltParser)，达到了图方法 (MSTParser) 的水平
2. **速度保持**：保持了转移分析的线性时间复杂度，甚至比传统转移分析器更快（因为省去了大量特征计算开销）

这打破了精度-速度的传统权衡。

## 7.6 本章小结

Chen & Manning (2014) 的神经依存分析器用分布式表示替代离散特征、用非线性网络替代线性分类器，在保持线性时间复杂度的同时，将分析精度提升到图方法的水平。三种嵌入（词、词性、依存标签）共同提供丰富的上下文信息。这项工作开创了神经句法分析的新时代。

# 8 依存分析的评估

## 8.1 UAS 与 LAS

依存分析的评估非常直观——逐条检查系统预测的依存弧是否与标准答案一致：

### 两种评估指标

- **UAS (Unlabeled Attachment Score)**：只看依存弧的方向是否正确（即每个词的中心词是否预测对了），不考虑关系标签

$$\text{UAS} = \frac{\text{正确预测中心词的词数}}{\text{总词数}}$$

- **LAS (Labeled Attachment Score)**：同时要求依存弧方向和关系标签都正确

$$\text{LAS} = \frac{\text{中心词和标签都正确的词数}}{\text{总词数}}$$

$\text{LAS} \leq \text{UAS}$ ，因为 LAS 的要求更严格。

例如对句子 “She saw the video lecture” (5 个词)，如果系统有 4 个词的中心词预测正确 (1 个错误)，但只有 2 个词的标签也完全正确，则  $\text{UAS} = 4/5 = 80\%$ ， $\text{LAS} = 2/5 = 40\%$ 。

## 8.2 本章小结

UAS 和 LAS 是依存分析的标准评估指标。UAS 只评估依存弧方向的正确性，LAS 同时要求方向和标签都正确。这种基于树库的定量评估方法是 1990 年代以来 NLP 领域的重要进步。

## 9 后续发展：从 SyntaxNet 到图方法

### 9.1 Google SyntaxNet / Parsey McParseface

2016 年，Google 在 Chen & Manning 的基础上进行了大规模改进，推出了 **SyntaxNet**（别名 Parsey McParseface）：

- **更深的网络**：使用更多层的神经网络和更大的向量维度
- **更优的超参数**：经过大规模调参
- **柱搜索** (Beam Search)：不再是纯贪心策略，而是同时维护多个候选分析路径，最终选择最优的一条
- UAS 从约 92% 提升到约 **94.6%**

Manning 提到，Google 将一个依存分析器做成了大新闻——在 Wired、VentureBeat 等科技媒体上广泛报道——这让他感到意外。起了一个搞笑的名字 (Parsey McParseface) 确实有助于获得媒体关注。

### 9.2 神经图方法依存分析 (Neural Graph-based Parsing)

除了转移方法，另一种重要的依存分析范式是**图方法** (Graph-based Parsing)：

#### 图方法的核心思想

1. 对句子中的**每对词**，计算一个依存弧的得分（共  $O(n^2)$  对）
2. 每个词需要选择一个中心词（或者是 ROOT），评估所有候选中心词的得分
3. 使用**最小生成树** (Minimum Spanning Tree) 算法从得分矩阵中找出得分最高的合法依存树（无环、连通、唯一根）

图方法的得分计算也可以使用神经网络。Manning 的团队将图方法与神经网络结合，构建了比 Parsey McParseface 更准确的分析器（UAS 再提升约 1%）。

这个神经图方法分析器被集成到了 Stanford 的开源 NLP 工具包 **Stanza** 中。

#### 转移方法 vs. 图方法

- **转移方法**：线性时间  $O(n)$ ，速度快，适合大规模处理；贪心决策可能导致错误传播
- **图方法**： $O(n^2)$  或  $O(n^3)$  时间，精度通常更高；考虑全局结构约束（树约束）
- 在传统和神经两个时代，图方法的精度都略高于转移方法
- 实践中两种方法都被广泛使用，取决于对速度和精度的需求

### 9.3 本章小结

Google 的 SyntaxNet 通过更深网络和柱搜索在转移方法上取得了进一步提升。图方法通过评估所有词对的依存可能性并求解最优树，提供了另一种精度更高的分析范式。两种方法的神经网络版本都显著优于传统符号化方法。

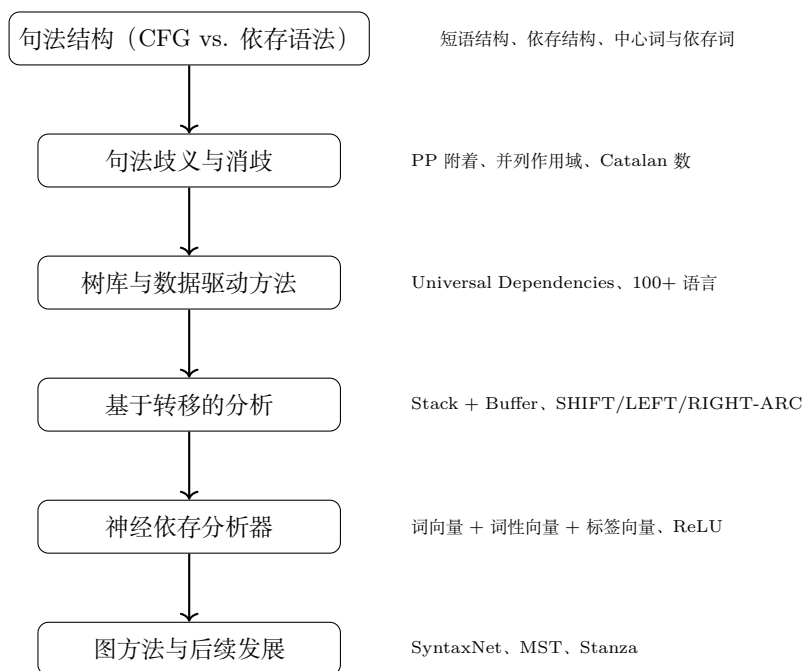
## 10 总结与延伸

### 10.1 讲者的核心总结

Chris Manning 在本讲中构建了从语言学概念到计算方法的完整链条：

1. **句法结构是理解语言的基础**：人类语言不是简单的词序列，而是具有层次结构的系统
2. **依存语法是高效的结构表示**：直接刻画词间修饰关系，历史悠久，在现代 NLP 中广泛应用
3. **歧义是核心挑战**：人类语言充满全局歧义，消歧需要世界知识和统计信息
4. **基于转移的分析是实用的解决方案**：线性时间复杂度，适合大规模处理
5. **神经网络带来了范式转换**：分布式表示解决了稀疏性问题，非线性分类器提升了精度，同时降低了计算开销

### 10.2 全课知识图谱



## 10.3 关键 Takeaways

### 五条核心原则

1. **依存语法 > 短语语法** (在 NLP 实践中): 依存结构直接表达词间关系, 标注效率高, 跨语言适用性强
2. **数据驱动 > 规则驱动**: 树库提供的统计信息远比手写规则更能有效消歧
3. **分布式表示 > 离散特征**: 词向量解决了稀疏性和不完备性问题, 同时降低了计算开销
4. **非线性分类器 > 线性分类器**: 神经网络的隐藏层能自动学习特征组合
5. **Transformer 中的隐式句法**: 后续课程将讨论, Transformer 语言模型虽然不显式构建依存树, 但其注意力机制隐式地学习了类似的结构信息

## 10.4 拓展阅读

- Nivre, J. (2005). Dependency grammar and dependency parsing. MSI report 5133(1).
- Chen, D. & Manning, C.D. (2014). A Fast and Accurate Dependency Parser using Neural Networks. *EMNLP 2014*. <https://nlp.stanford.edu/pubs/emnlp2014-depparser.pdf>
- Andor, D. et al. (2016). Globally Normalized Transition-Based Neural Networks. *ACL 2016*. (Google SyntaxNet / Parsey McParseface)
- Dozat, T. & Manning, C.D. (2017). Deep Biaffine Attention for Neural Dependency Parsing. *ICLR 2017*. (神经图方法)
- Universal Dependencies 项目: <https://universaldependencies.org/>
- Stanford Stanza NLP 工具包: <https://stanfordnlp.github.io/stanza/>
- Jurafsky, D. & Martin, J.H. *Speech and Language Processing*, Chapter 18: Dependency Parsing. <https://web.stanford.edu/~jurafsky/slp3/>